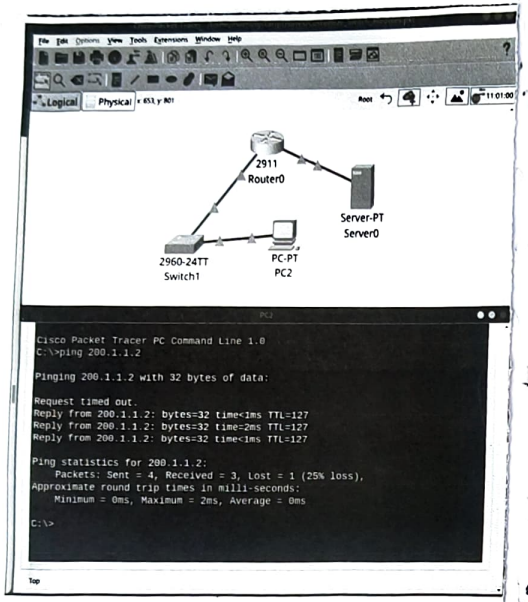


Ex no: 11 — Date: 12/9/26

Routing at Network Layer

a) Simulate Static Routing Protocol Configuration using Cisco Packet Tracer.

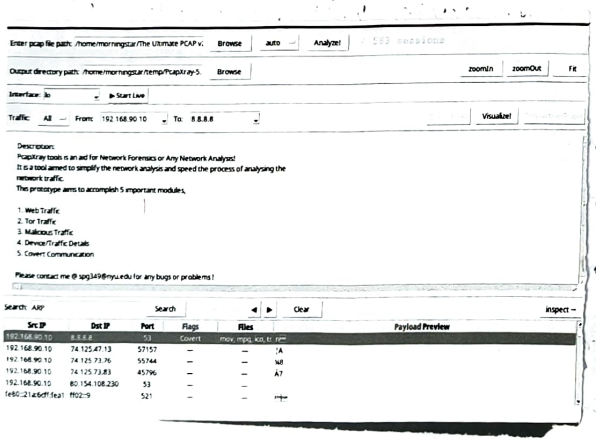
* Aim:

To simulate and configure static Routing Protocol using Cisco Packet Tracer and observe packet routing between different networks.

* Procedure:

1) Open Cisco packet Tracer.
2) place three routers, two switches and four PCs.
3) Connect the devices using appropriate Ethernet cabel
4) Enable the router interface using the no shutdown command.
5) Configure static routes on the routers using IP route command.
6) Configure a primary route and backup route using different administrative distance.
7) Verify using routing table show ip route static.
8) Test Connectivity between Pcs using the ping command
9) Use Tracert <destination-IP> to observe destination
10) Simulate the failure the main route and verify that the backup route is used.

```
~
→ tshark -r "$F" -Y "dns.flags.response==0" -T fields -e ip.src -e dns.qry.name -e dn
s.qry.type | sort | uniq -c | sort -rn | head -20
     53 172.16.81.99    v10.events.data.microsoft.com   28
     44 172.16.81.99    v10.events.data.microsoft.com   1
     25         iTunes_Ctrl_73F532408528F239._dacp._tcp.local   255
     25 192.168.7.26    iTunes_Ctrl_73F532408528F239._dacp._tcp.local   255
     24 172.16.81.99    settings-win.data.microsoft.com 28
     21 172.16.81.99    wpad.ad2024.weberlab.de 1
     20 172.16.81.99    wpad.ad2024.weberlab.de 28
     18 172.16.81.99    settings-win.data.microsoft.com 1
     18 172.16.81.99    cp601.prod.do.dsp.mp.microsoft.com      28
     17 172.16.81.99    3.tlu.dl.delivery.mp.microsoft.com      28
     17 172.16.81.99    3.tlu.dl.delivery.mp.microsoft.com      1
     15 192.168.121.2   blog.webernetz.net      1
     15 192.168.110.9   blog.webernetz.net      1
     15 172.16.81.99    vm39-ad2024.ad2024.weberlab.de  1
     15 172.16.81.99    cp601.prod.do.dsp.mp.microsoft.com      1
     14         _homekit._tcp.local,_airplay._tcp.local,_sleep-proxy._udp.local12,12,
12
     14 85.215.94.29    router1.weberlab.de     1
     14 192.168.7.26    _homekit._tcp.local,_airplay._tcp.local,_sleep-proxy._udp.loc
al 12,12,12
     13 172.16.81.99    g.live.com      28
     12 172.16.81.99    g.live.com      1
```



* Result:

Thus, the Static Routing was successfully simulated and configured using cisco packet Tracer through primary and backup routes was verified.

b) Simulate RIP using Packet Tracer

* Aim:

To simulate RIP using Cisco Packet Tracer and observe dynamic routing between different network.

* Procedure:

1) Open Cisco Packet Tracer.
2) Place three router and Two Pcs.
3) Connect three router in a triangular topology.
4.) Connect the Pc0 to Router 0 and Pc1 to Router$_2$.
5) Configure the IP addresses of all Router interface and.
6) Enable the interface using no shutdown.
7.) Configure RIP on each router using router rip
8.) Advertise the directly connected networks using the network command.
9.) verify the routing tables using show ip route.
10.) Test Connectivity between Pc0 and Pc1 using
11.) Use Tracert to observe the route selected by RIP.
12.) Disconnected one route and observe the RIP automated selected another available route.

* Result:

Thus, RIP was successfully simulated using cisco packet Tracer, and Dynamic route select and automatic rerouting were verified.

```
Socket s = ss.accept();
BufferedReader in = new BufferedReader(
    new InputStreamReader(s.getInputStream()));
PrintWriter out = new PrintWriter(s.getOutputStream(), true);
String msg;
while ((msg = in.readLine()) != null) {
    out.println("Echo:" + msg);
    if (msg.equalsIgnoreCase("bye")) break;
}
s.close();
ss.close();
}}
import java.io.*;
import java.net.*;
public class TCPEchoClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        BufferedReader in = new BufferedReader(
        new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader kb = new BufferedReader(
            new InputStreamReader(System.in));
        String msg;
        while ((msg = kb.readLine()).equalsIgnoreCase("bye")) {
            out.println(msg);
```

```
System.out.println(readline());
}
s.close();
}
}
```

* Result:

Thus the TCP Echo client server application was successfully implemented and the client successfully received the echoed messages from the server.

b) Implementation echo client server using UDP socket.

* Aim:

To implement an echo client server application UDP socket and verify communication between the client and server.

* Procedure:

1) Create a UDP Echo Server using Datagram socket and Datagram Packet.
2) Assign an port no to the UDP server.
3) Compile and execute the UDP server program.
4) Create a UDP echo client using datagram socket and Datagrams..
5) Specify the server address and port no.
6) Enter a message through the client.
7) The client sends the message as a UDP datagram to server.
8) The server recieves the datagrams and sends.
9) Display the received messages.
10) Enter bye to terminate the communication.

* Program:

```
import java.net.*;
public static void main ( String [] args throws Exception{
```

```
DatagramSocket s = new DatagramSocket(6000);
byte [] b = new byte [1024];
while (true){
  DatagramPacket p = new DatagramPacket (b, b.length);
import . java . net . *;
import . java . io . *;
Public class UDPEchoClient {
Pubic static void main (String [] args) throws exception{
DatagramSocket S = new DatagramSocket()
InternetAddress a = InetAddress . getByName . ("localHost");
 BufferedReader . bb = new BufferedReader(
     new InputStreamReader (System . in));
String msg;
while (! (msg = (b . readLine()) . equals Ignore Case ("bye")){
  byte [] b = msg . getBytes();
  S . send (new DatagramPacket (b, b.length , 9, 6001));
    byte [] r = new byte [1024];
      S . receive (r);
      System . out . println (new String (P . getData() -0, r . get
                                          Length()));
  S . close ();
Result!
```

Thus, the UDP Echo client . server application was successfully implement and the client successfully received the enclosed message from the server.

6) To implement actual client - server application using sockets.

* Aim:
To implement a client - server application using socket and enable communication between multiple clients through server.

* Procedure:
1) Create a chat server program socket programming.
2) ~~Create the~~ Start & run the client server.
3) Create the client program and connect it to the server.
4) Run two or more client program in seperate terminals.
5) Enter a name for each client.
6) Send messages from client to server.
7) The server broadcast the message to the connected client.
8) Verify that the client can exchange message through server.
9) Enter 1 quit to terminate a client connection.

* Program:

```
import java.io.*;
import java.net.*;
import java.util.*;
```

```
Public class chatd Server{
 Static . Set <printwriter> c = new HashSet >();
public static void main (String[]) throws Exception{
 Server Socked s = new Server Socked (5000);
 System.out.println ("Chat Server. Started");
while (true){
   Socked P = s.accept ())
   new Thread () ->Chat(P)). start();
 }}
Static void .Chat (socket s){
 try {
   BufferReader in = new BufferedReader.C
 new input StreamReader ( s.getInputStream ()));
 printwriter out = new printwriter( s.getoutputstream.out)
 ( add (out);
String in;
while ((in = in . readline()) != null()
      for (printwriter P : C ). P. Println (in);
   C. remove (out);
    S.close ();
   } catch (Exception e) {}
 }
}
```

```
import java.io.*;
import java.net.*;
public class chatclient {
  Public Static void main (String[] a) throws Exception
    Socket S = new Socket ("127.0.0.1", 5000);
  BufferedReader in = new BufferedReader (new InputStream
      Stream Reader (S.getInputStream()));
  new Thread (()->{
  try {
    String m)
    while ((m = in.readline()) != null) {
    System.out.println (m);
    Catch (Exception e) { }
  }) Start ();
  String m;
  while ((m=kb.readline()) != null){
    out.println(m);
    if (m.equal.Ignore ("quit")) break;
  }
  S.close();
  }
}
```

* Result:

Thus, the chat client server application was successfully implemented and messages were successfully exchanged b/w multiple client through the server.

* Outputs:

TCP Echo Client-Server

Message : Hello
Echo : Hello

UDP Echo Client-Server

Message : UDP
Echo : UDP



Exno: 12a
Date: 17/9/26

ECHO CLIENT - SERVER USING TCP/UDP SOCKETS

* Aim: To implement an echo client-server application using TCP and UDP sockets.

* Procedure:

1. Create and compile the TCP and UDP server/client programs.
2. Start the appropriate server in one terminal.
3. Start its client in another terminal and enter a test message.
4. Verify that the server returns the received message with an echo prefix.

* Program:

-> TCP Echo Server - TCPEchoServer.java

```
import java.io.*;
import java.net.*;

public class TCPEchoServer {
    public static void main(String[] args) throws IOException {
        int port = args.length > 0 ? Integer.parseInt(args[0]) :
                                                    8000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo Server listening on port "
                                                    + port);
        while (true) {
            try (Socket clientSocket = new ServerSocket(port)) {
```

```
try(Socket clientSocket = serverSocket.accept()){
    System.out.println("Client connected! "+ clientSocket.
                                    getRemoteSocketAddress());

    BufferedReader in = new BufferedReader(
        new InputStreamReader(clientSocket.getInputStream()));
    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(),
                                    true);

    String line;
    while((line = in.readline()) != null){
      System.out.println("Received: " + line);
      out.println("Echo: " + line);
      if(line.equalsIgnoreCase("bye")) break;
    }
     System.out.println("Client disconnect.");
    } catch(IOException e){
    System.out.println("Connection error: " + e.getMessage());
    }
  }
 }
}
```

→ TCP Echo Client - TCPEchoClient.java

```
import java.io.*;
import java.net.*;

public class TCPEchoClient {
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
```

```
int port = args.length > 1 ? Integer.parseInt(args[1]):5000;

try (Socket socket = new Socket(host, port);
  PrintWriter out = new Printer(socket.getOutputStream(), true);
  BufferedReader in = new BufferedReader(
      new InputStreamReader(socket.getInputStream()));
  BufferedReader console = new BufferedReader(
      new InputStreamReader(System.in))) {
  System.out.println("Connected to TCP Echo Server at "+host+
                                        ":"+port);
  System.out.println("Type messages (type 'bye' to quit):");
  String message;
}
  System.out.println("Connection closed");
}
```

```
* UDP Echo Server - UDPEchoServer.java
import java.net.*;
import java.nio.charset.StandardCharsets;
public class UDPEchoServer {
    public static void main(String[] args) throws
                                        Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]):
                                        6000;
        byte[] buffer = new byte[1024];
```

```
try(DatagramSocket socket = new DatagramSocket(port)){
    System.out.println("UDP Echo Server listening on port"+
                        port);
    while(True){
    DatagramPacket request = new DatagramPacket(buffer,
                                buffer.length);
     Socket.receive(request))
     String message = new String( request.getData(),0,
          request.getLength(), StandardCharsets.UTF_8)
     System.out.print("Received : "+message);
     Socket.send(response);
    }
}
```

* UDP EchoClient - UDPEchoClient.java

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UDPEchoClient{
    public static void main(String[] args) throws IOException{
        String host = args.length>0?args[0]:"localhost";
        int port = args.length>1?Integer.parseInt(args[1])
                                :6000;
        InetAddress address = InetAddress.getByName(host);
    try(DatagramSocket socket = new DatagramSocket();
        BufferedReader console = new BufferedReader(
            new InputStreamReader(System.in))){
```

```
Socket.setSoTimeout(5000);
System.out.println("UDP EchoClient -> " + host + ":" + port);
System.out.println("Type messages (type 'bye' to quit):");

String message;
while ((message = console.readLine()) != null) {
    byte[] data = message.getBytes(StandardCharsets.UTF_8);
    socket.send(new DatagramPacket(data, data.length,
     address, port));
    byte[] buffer = new byte[1024];
    DatagramPacket reply = new DatagramPacket(buffer, buffer.length);
    try {
      socket.receive(reply);
      System.out.println("server: " + new String(reply.getData(), 0,
    } catch (SocketTimeoutException e) {
        System.out.println("(no response received (timeout))");
    }
    if (message.equalsIgnoreCase("bye")) break;
}
```

Result:

Echo client-server communication was implemented and tested using TCP and UDP sockets.

Chat Program Series Side

You : Hi

Server : ~~I am~~ Hello

You : How are you?

Server : I am healthy

You : /quit

Chat Program - Client Side

Client : Hi

You : Hello

Client : How are you?

You : I am healthy.

Ex.no: 12b

Date:

CHAT PROGRAM USING SOCKET PROGRAMMING

* Aim: To implement a client-server chat program using socket programming.

* Procedure:

1. Compile the chat server and client.
2. Start the server, then launch two or more clients in separate terminals.
3. Enter client names and exchange messages.
4. Enter /quit to disconnect a client.

* Program:

-> ChatServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;
public class ChatServer {
    private static final int PORT = 8000;
    private static final Set<ClientHandler> clients =
    Collections.synchronizedSet(new HashSet<>());
public static void main(String[] args) throws IOException{
    try (ServerSocket server = new ServerSocket(PORT)){
    System.out.println("Chat Server started on port "+PORT);
while(true){
    Socket socket = server.accept();
```

```
        ClientHandler handler = new ClientHandler(socket);
        clients.add(handler);
        new Thread(handler).start();
    }
}

private static void broadcast(String message, ClientHandler
                                                sender) {
    synchronized(clients) {
        for (ClientHandler client : clients) {
            if (client != sender) client.send(message);
        }
    }
}

public void run() {
    try {
        in = new BufferedReader(new InputStreamReader(socket.
                                                getInputStream()));
        out = new PrintWriter(socket.getOutputStream(), true);
        if (suppliedName != ~~in.readLine()~~, NULL && !suppliedName
                                .isBlank()) name = suppliedName;

        broadcast(name + " has joined the chat.", this);
    } catch (IOException e) {
        System.out.println("Connection error: " + e.getMessage());
    } finally {
        clients.remove(this);
        broadcast(name + " has left the chat.", this);
        try { socket.close(); } catch (IOException ignored) {}
    }
}
```

* ChatClient.java

```
import java.io.*;
import java.net.*;
public class ChatClient {
    private static final String SERVER_ADDRESS = "127.0.0.1";
    private static final int SERVER_PORT = 5000;
    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket(SERVER_ADDRESS,
                                        SERVER_PORT)) {

        Thread listener = new Thread(() -> {
            try {
                String message;
                while ((message = in.readLine()) != null) System.out.
                                                println(message);
            } catch (IOException e) {
                System.out.println("Disconnected from server.");
            }
        });
    }
}
```

* Result:

A socket-based multi-client chat application was implemented and tested.